

settlement_app

How the Files Connect — an architecture & wiring map

Layered PySide6 desktop app • generated from the live source tree

This document traces how the source files in `src/settlement_app/` depend on and call one another. It follows four threads: (1) the layered structure, (2) the startup & dependency-injection chain, (3) the one fully-wired runtime data flow (Excel download), and (4) a complete file-to-file import reference.

1 The four layers

The project follows a clean / hexagonal layering. Dependencies point inward: **presentation** and **infrastructure** depend on **application**, which depends on **domain**. `domain` imports nothing from the other layers.

Layer	Folder	Holds	Depends on
Entry	<code>__main__.py</code>	Process entry point; builds <code>QApplication</code> , loads settings, opens <code>MainWindow</code>	presentation, infrastructure
Presentation	<code>presentation/</code>	<code>MainWindow</code> , 7 pages, dialogs, reusable widgets, theme	application, infrastructure (settings/scrapers)
Application	<code>application/</code>	Services, use-cases, DTOs, the Scraper Protocol, <code>RunRegistry</code>	domain only
Infrastructure	<code>infrastructure/</code>	Settings loader, Postgres pool, Playwright scrapers	application (protocol/DTOs), domain
Workers	<code>workers/</code>	<code>ScraperWorker</code> (<code>QRunnable</code>) bridging scrapers to the GUI thread	application, infrastructure
Domain	<code>domain/</code>	Entities, value objects, exceptions; pure data + rules	nothing (innermost)

Key idea: the seam between layers is always an **interface or DTO**, never a concrete class reaching across. The most important seam is `application/services/scraper_protocol.py` (the *Scraper Protocol*) and the Qt-free `application/dto/run_state.py` (`RunState`) — both are designed to be passed safely across threads.

2 Startup & dependency-injection chain

Nothing wires itself. `__main__.main()` bootstraps Qt, then `MainWindow.__init__` acts as the **composition root**: it creates the shared services once and injects them into each page through constructors.

```
__main__.main()
■ load_settings() → infrastructure/settings/app_settings.py
■ app.setStyleSheet(GLOBAL_QSS) → presentation/theme.py
■ MainWindow() → presentation/main_window.py (composition root)
■ RunRegistry(parent=self) # ONE shared instance, on the GUI thread
■ load_settings() → AppSettings
■ DashboardService(registry) # application/services/*
■ ResultsService(registry)
■ builds 7 pages into a QStackedWidget, injecting deps
```

Who gets injected with what

#	Page (presentation/widgets/pages/)	Constructor receives	So it can...
0	dashboard_page.py	DashboardService, RunRegistry	show KPIs + live recent activity
1	data_import_page.py	AppSettings	read DB/storage config for the import view
2	excel_download_page.py	RunRegistry, AppSettings	dispatch scraper jobs + show live status
3	settlement_run_page.py	ResultsService	list settlement targets / failures
4	monitoring_page.py	RunRegistry	subscribe to run list + log stream
5	results_page.py	ResultsService	show result KPIs + branch table
6	settings_page.py	AppSettings	render current config values

Cross-page wiring done in MainWindow: monitoring_page.back_requested → _on_back, and excel_page.view_logs_requested → _on_view_logs_requested → monitoring_page.filter_to_run(). The single RunRegistry is the shared bus both the Excel page and the Monitoring page subscribe to.

3 The one fully-wired runtime flow: Excel download

This is the path to study, because every other page is meant to copy its shape. It is the only flow that runs end-to-end today, and it shows how a button press crosses the GUI thread → a worker thread → back to the GUI thread using Qt signals.

```
ExcelDownloadPage._on_run_source("coupang" / "baemin") [GUI thread]
→ registry.register_run(...) → RunState (application/dto/run_state.py)
→ CoupangScraper(settings) / BaeminScraper(settings)
( satisfies Scraper Protocol → application/services/scraper_protocol.py )
→ make_worker(...) → workers/scraper_worker.py (QRunnable + RunWorkerSignals)
→ QThreadPool.globalInstance().start(worker)

worker.run() [POOL thread]
→ scraper.download_excel(...) → Playwright drives the browser
→ signals.progress / log / completed / failed .emit(...)

RunWorkerSignals (created on GUI thread)
→ Qt QueuedConnection → RunRegistry.on_worker_* slots [back on GUI thread]
→ registry re-emits: run_added / run_status_changed /
run_log_appended / run_progress_updated
→ ExcelDownloadPage slots (status + progress cards)
→ MonitoringPage (run list + live log view)
```

Why it is thread-safe

RunWorkerSignals is a QObject created on the GUI thread *before* the worker is submitted. When the pool thread calls `.emit()`, Qt detects the receiver lives on a different thread and automatically marshals the call via QueuedConnection — no manual `invokeMethod` needed. RunState carries no Qt objects, so it is safe to hand between threads (and, later, to serialize over a network).

4 Inside the scraper layer

The page only knows the Scraper Protocol. Behind it, two adapter classes wrap verbatim automation cores plus their OTP helpers.

File (infrastructure/scrapers/)	Role	Pulls in
scraper_protocol.py (in application/)	Defines Scraper Protocol + ScraperProgress callback	—
coupang_scraper.py	Sync adapter implementing the protocol	coupang_core, coupang_imap_otp, AppSettings
coupang_core.py	Verbatim Playwright sync automation	coupang_imap_otp (relative import)
coupang_imap_otp.py	Email OTP retrieval	imap-tools, keyring
baemin_scraper.py	Sync adapter around the async core	baemin_core, AppSettings
baemin_core.py	Verbatim Playwright async automation	server/sms_otp_waiter, baemin_excel_extractor
server/sms_otp_waiter.py	HTTPServer receiving SMS JSON from a phone	stdlib (http.server, threading)
baemin_excel_extractor.py	Downloads/extracts the Baemin Excel files	Playwright

All of these relative imports were verified to resolve against files that exist on disk; `python -m compileall src` passes cleanly.

5 Supporting layers and where they plug in

Module	Imported by	Purpose / connection
infrastructure/settings/app_settings.py	__main__, MainWindow, every page, both scrapers, postgres.py	AppSettings dataclasses + load_settings(); the config hub read everywhere
application/dto/run_state.py	run_registry, scraper_worker, excel_download_page, monitoring_page, dashboard/results service	RunState/RunStatus — the object that travels the live flow
application/services/run_registry.py	MainWindow, excel/monitoring pages, dashboard/results service, scraper_worker	QObject signal bus; the single shared coordination point
infrastructure/db/postgres.py	(none yet) imports app_settings	get_pool() lazy psycopg pool; repositories are not built yet
presentation/theme.py	__main__, main_window, every widget & dialog	COLOR dict + GLOBAL_QSS styling
presentation/widgets/_scaffold.py + badge/buttons/cards/table/misc	all 7 pages and the dialogs	shared building blocks every page composes from
domain/entities/*	use_cases (download_job), DTOs conceptually	pure data: Branch, Rider, DownloadJob, SettlementRun, ScraperHost

6 Connected vs. declared-but-empty

The injection wiring is complete and correct everywhere, but most implementations behind the seams are still stubs. An honest status of the edges:

State	What it covers
Live	Excel download → RunRegistry → ScraperWorker → scrapers → Monitoring. Settings/theme/widget composition. RunRegistry signal bus.
Wired, stub body	DashboardService, ResultsService injected into pages but return hardcoded/empty data. Pages render static rows.
Declared, not built	use_cases (run_settlement, enqueue_download_run raise NotImplementedError); infrastructure/repositories/ is empty; settlement math is delegated out.

In short: the dependency graph (who imports / injects whom) is sound and matches this map. What remains is filling in the service and repository bodies behind the existing seams.

Reference: full file-to-file import edges follow on the next page.

Appendix Complete internal import edges

Every intra-package import in `src/settlement_app/`, as extracted from source. Read each row as “left file imports from right module”.

File	Imports from
<code>__main__.py</code>	<code>infrastructure/settings/app_settings</code> (<code>load_settings</code>)
<code>__main__.py</code>	<code>presentation/main_window</code> (<code>MainWindow</code>)
<code>__main__.py</code>	<code>presentation/theme</code> (<code>GLOBAL_QSS</code>)
<code>presentation/main_window.py</code>	<code>application/services/dashboard_service</code> , <code>results_service</code> , <code>run_registry</code>
<code>presentation/main_window.py</code>	<code>infrastructure/settings/app_settings</code> (<code>load_settings</code>)
<code>presentation/main_window.py</code>	<code>presentation/theme</code> (<code>COLOR</code>)
<code>presentation/main_window.py</code>	<code>presentation/widgets/pages/*</code> (all 7 pages)
<code>application/services/dashboard_service.py</code>	<code>application/dto/dashboard_dto</code> , <code>run_state</code> ; <code>services/run_registry</code>
<code>application/services/results_service.py</code>	<code>application/dto/run_state</code> , <code>settlement_dto</code> ; <code>services/run_registry</code>
<code>application/services/download_service.py</code>	<code>application/dto/download_dto</code>
<code>application/services/run_registry.py</code>	<code>application/dto/run_state</code>
<code>application/use_cases/enqueue_download_run.py</code>	<code>domain/entities/download_job</code>
<code>workers/scrapper_worker.py</code>	<code>application/services/run_registry</code> , <code>scrapper_protocol</code> ; <code>application/dto/run_state</code>
<code>infrastructure/db/postgres.py</code>	<code>infrastructure/settings/app_settings</code>
<code>infrastructure/scrapers/coupang_scraper.py</code>	<code>application/services/scrapper_protocol</code> ; <code>scrapers/coupang_core</code> , <code>coupang_imap_otp</code> ; <code>settings</code>
<code>infrastructure/scrapers/baemin_scraper.py</code>	<code>application/services/scrapper_protocol</code> ; <code>scrapers/baemin_core</code> ; <code>settings</code>
<code>infrastructure/scrapers/coupang_core.py</code>	<code>scrapers/coupang_imap_otp</code> (relative)
<code>infrastructure/scrapers/baemin_core.py</code>	<code>scrapers/server/sms_otp_waiter</code> , <code>baemin_excel_extractor</code> (relative)
<code>presentation/widgets/pages/dashboard_page.py</code>	<code>services/dashboard_service</code> , <code>run_registry</code> ; <code>dialogs/run_all_confirm</code> ; <code>widgets/*</code>
<code>presentation/widgets/pages/excel_download_page.py</code>	<code>dto/run_state</code> ; <code>widgets/*</code> ; (lazy) <code>scrapers/*</code> , <code>workers/scrapper_worker</code> , <code>settings</code>
<code>presentation/widgets/pages/monitoring_page.py</code>	<code>dto/run_state</code> ; <code>widgets/*</code> ; (typing) <code>services/run_registry</code>
<code>presentation/widgets/pages/results_page.py</code>	<code>dialogs/send_complete</code> ; <code>widgets/*</code> ; (typing) <code>services/results_service</code>
<code>presentation/widgets/pages/settlement_run_page.py</code>	<code>dialogs/settlement_mapping</code> ; <code>widgets/*</code> ; (typing) <code>services/results_service</code>
<code>presentation/widgets/pages/settings_page.py</code>	<code>settings/app_settings</code> ; <code>widgets/*</code>
<code>presentation/widgets/pages/data_import_page.py</code>	<code>widgets/*</code> ; (typing) <code>settings/app_settings</code>

File	Imports from
presentation/dialogs/*.py	dialogs/_base_dialog; presentation/theme; widgets/badge, buttons, table
presentation/widgets/* (badge/cards/misc)	presentation/theme (COLOR)